

# Conv-ChArT — receptive field of the bottleneck

Computed from the live module list, not a hand-written layer table

## The receptive field, analytically

For a stack of  $L$  layers, layer  $l$  having kernel  $k_l$ , stride  $s_l$  and dilation  $d_l$ , the receptive field  $r$  of one output unit measured in **input pixels**, and the jump  $j$  (input pixels per output step), are

$$r = 1 + \sum_{l=1}^L \left( d_l (k_l - 1) \prod_{i=1}^{l-1} s_i \right), \quad j = \prod_{l=1}^L s_l$$

Read it as: each layer contributes its own **dilated** kernel extent  $d_l(k_l - 1)$ , magnified by the total downsampling **accumulated before it**,  $\prod_{i<l} s_i$ . A layer late in the stack is worth far more reach than an early one, which is why dilating at the deepest stage is the cheap way to buy receptive field — and why a stride-2 pool doubles the value of every layer that follows it.

Equivalently, as the recursion actually evaluated below (Dumoulin & Visin 2016), starting from  $r = 1, j = 1$ :

$$k_{\text{eff}} = d(k - 1) + 1, \quad r \leftarrow r + (k_{\text{eff}} - 1)j, \quad j \leftarrow j \cdot s$$

Dilation inflates the kernel **without adding parameters** — that is the whole appeal, and the reason the pair was cheap enough to leave in.

## Worked, for the deepest stage

Every conv here is  $k = 3, s = 1$ ; the three pools are  $k = 2, s = 2$ . By the time the signal reaches **e4** the accumulated stride is  $\prod_{i<l} s_i = 8$ , so each **e4** layer contributes  $8 d_l (k_l - 1) = 16 d_l$  input pixels:

$$\underbrace{16 \times 1}_{\text{e4 conv 1}} + \underbrace{16 \times 1}_{\text{e4 conv 2}} + \underbrace{16 \times 2}_{\text{dilated } d=2} + \underbrace{16 \times 4}_{\text{dilated } d=4} = 32 + 96 \text{ px}$$

The undilated pair of **e4** buys 32 px; **the dilated pair buys the other 96 px** — which is exactly the difference in the table below, and what 1,180,672 parameters are being spent on.

## Result at the bottleneck (8x downsample, 60 x 80 tokens)

|                                 | theoretical $r$ | jump $j$ | Luo effective radius |
|---------------------------------|-----------------|----------|----------------------|
| <b>with</b> dilated (2, 4) pair | <b>164 px</b>   | 8        | ~45 px               |
| <b>without</b> the pair         | <b>68 px</b>    | 8        | ~21 px               |
| difference                      | 96 px           | –        | ~25 px               |

**Theoretical  $r$  is an upper bound**, not what the network uses. Luo et al. (2016) show the influence of input pixels on an output is approximately Gaussian, and that for a stack of  $n$  layers the effective radius grows as

$O(\sqrt{n})$  rather than linearly with depth — so the usable extent is a shrinking fraction of  $r$ . With  $n = 13$  layers to the bottleneck, the effective radius is roughly  $r/\sqrt{n} \approx 45$  px.

## Why this is the design question

The identity read needs to see an **adjacent marker**: the marker centre nearest a given inner corner sits about  $0.85s$  away, where  $s$  is the board square size. Over the trained envelope  $s \in [12, 128]$  px that distance is **10 to 109 px**.

- At  $s = 12$  px the neighbour is 10 px away — comfortably inside even the effective radius. Convolutions alone suffice, which is exactly what the A1 ablation measured: conv-only is *equal or better* at far range.
- At  $s = 128$  px the neighbour is 109 px away — outside the effective radius (45 px) and, without the pair, outside it by a wide margin. Local evidence cannot answer the question, and the bottleneck attention (global by construction) is the only mechanism that can. A1 measures the cost of removing it: **99.20% vs 91.82% ID accuracy at  $s = 128$** .

## The dilation question

The dilated pair buys **96 px of theoretical reach for 1,180,672 parameters** — 25.1% of the detector, 0.75x the cost of the attention that superseded it. It is inherited from the pre-transformer design (Rev B / DS-02), where the dilated cascade *was* the board-scale context mechanism; the Rev G review marked it moot once the MHSA bottleneck landed, and it was never removed.

Two readings, and the ablation `configs/abl_nodilate.yaml` decides between them:

1. **Redundant** — attention already provides unbounded reach, so the extra 96 px is paid for and unused.
2. **Actively harmful** — RoPE attention discriminates tokens by content *and* relative position; a wide dilated aggregation makes neighbouring tokens more alike and erodes the local distinctiveness the attention depends on.

Note that even *with* the pair the effective radius (45 px) does not reach an adjacent marker at large  $s$  — so the dilation does not solve the close-range problem either. That is the argument for its removal being free.

## Layer-by-layer, with the dilated pair

| layer        | k | s | d | k_eff | r (px)     | j |
|--------------|---|---|---|-------|------------|---|
| e1.0.0 k3    | 3 | 1 | 1 | 3     | <b>3</b>   | 1 |
| e1.1.0 k3    | 3 | 1 | 1 | 3     | <b>5</b>   | 1 |
| pool -> e2   | 2 | 2 | 1 | 2     | <b>6</b>   | 2 |
| e2.0.0 k3    | 3 | 1 | 1 | 3     | <b>10</b>  | 2 |
| e2.1.0 k3    | 3 | 1 | 1 | 3     | <b>14</b>  | 2 |
| pool -> e3   | 2 | 2 | 1 | 2     | <b>16</b>  | 4 |
| e3.0.0 k3    | 3 | 1 | 1 | 3     | <b>24</b>  | 4 |
| e3.1.0 k3    | 3 | 1 | 1 | 3     | <b>32</b>  | 4 |
| pool -> e4   | 2 | 2 | 1 | 2     | <b>36</b>  | 8 |
| e4.0.0.0 k3  | 3 | 1 | 1 | 3     | <b>52</b>  | 8 |
| e4.0.1.0 k3  | 3 | 1 | 1 | 3     | <b>68</b>  | 8 |
| e4.1.0 k3 d2 | 3 | 1 | 2 | 5     | <b>100</b> | 8 |
| e4.2.0 k3 d4 | 3 | 1 | 4 | 9     | <b>164</b> | 8 |

## Layer-by-layer, without it

| layer      | k | s | d | k_eff | r (px)    | j |
|------------|---|---|---|-------|-----------|---|
| e1.0.0 k3  | 3 | 1 | 1 | 3     | <b>3</b>  | 1 |
| e1.1.0 k3  | 3 | 1 | 1 | 3     | <b>5</b>  | 1 |
| pool -> e2 | 2 | 2 | 1 | 2     | <b>6</b>  | 2 |
| e2.0.0 k3  | 3 | 1 | 1 | 3     | <b>10</b> | 2 |
| e2.1.0 k3  | 3 | 1 | 1 | 3     | <b>14</b> | 2 |
| pool -> e3 | 2 | 2 | 1 | 2     | <b>16</b> | 4 |
| e3.0.0 k3  | 3 | 1 | 1 | 3     | <b>24</b> | 4 |
| e3.1.0 k3  | 3 | 1 | 1 | 3     | <b>32</b> | 4 |
| pool -> e4 | 2 | 2 | 1 | 2     | <b>36</b> | 8 |
| e4.0.0 k3  | 3 | 1 | 1 | 3     | <b>52</b> | 8 |
| e4.1.0 k3  | 3 | 1 | 1 | 3     | <b>68</b> | 8 |